STREAMPAY

Contract Specification & Full System Prompt for AI Agent
Non-Custodial Per-Second Streaming Payments on Stellar Soroban
Drips Wave 11 — Maintainer Application Package

Document Role: This is Document 1 of 2 for StreamPay. An AI coding agent must read this entire document, then execute the numbered build sequence exactly as written. No questions, no assumptions outside this spec. Document 2 covers the frontend/app monorepo.

1. Project Identity & Positioning

Name: StreamPay
Tagline: Continuous, non-custodial value transfer on Stellar — per second, pausable, cancellable.
One-sentence pitch: StreamPay lets any party open a streaming payment of SEP-41 USDC (or any Token interface asset) that accrues every ledger; either party can pause, resume, update the rate, or cancel; anyone can execute the accrued transfer trustlessly.
Why it wins Wave 11: Streaming is still white-space on Drips (~843 approved repos). Most payment projects are one-shot escrow or payment links. Continuous streaming is a core primitive for salaries, subscriptions, creator payouts, and agentic commerce. Stellar's native per-ledger timing + low fees make it uniquely suited. The contract is single-responsibility, fully deterministic, and produces clear events for indexers — perfect for a solo maintainer.

2. Exact Scope (What Ships in Wave 11)

In Scope (MVP — must be complete and demoable):

- One Soroban contract: StreamPay (Rust, soroban-sdk 30.0.0-rc.1 or latest stable 30.x).
- Initialize with admin + token address (SEP-41 Token interface).
- create_stream(payer, payee, rate_per_second) → stream_id (u64).
- pause / resume / update_rate / cancel by authorized party (payer or payee rules defined).
- execute_payment(stream_id) — anyone can call; transfers accrued amount; updates last_paid_ledger.
- View functions: get_stream, amount_owed.
- Events for every state change.
- Persistent storage with proper TTL extension.
- No floats. All math in i128 (stroops or token base units).
- require_auth() on every mutating entry point that changes ownership or balances.
- ContractError enum for every failure mode.

Explicitly Out of Scope (do not implement):

- Multi-token streams in one call (one token per stream is enough).
- Automatic clawback or freeze (token issuer handles that).
- On-chain scheduling of execute_payment (off-chain keepers or frontend do it).
- Confidential / private amounts (that is PrivateRail).
- Governance, fees to protocol, or treasury.
- Cross-chain bridges.

3. Contract Architecture

Single contract, single responsibility. No external contract dependencies except the TokenClient for the configured SEP-41 token. Storage is keyed by stream_id. Admin can only set the token once at initialize and later (optional) update a protocol fee rate if desired — but MVP has zero protocol fee.

3.1 Storage Layout

```
#[contracttype]
#[derive(Clone)]
pub enum DataKey {
    Admin,                    // Address
    Token,                    // Address of SEP-41 token
    NextStreamId,             // u64
    Stream(u64),              // Stream struct
}

#[contracttype]
#[derive(Clone)]
pub struct Stream {
    pub id: u64,
    pub payer: Address,
    pub payee: Address,
    pub rate_per_second: i128,   // token base units per second
    pub start_ledger: u32,
    pub last_paid_ledger: u32,
    pub paused: bool,
    pub cancelled: bool,
    pub total_paid: i128,
}
```

3.2 Core Math (Deterministic)

amount_owed = rate_per_second * (current_ledger – last_paid_ledger) when not paused and not cancelled.
When paused, accrual stops at the ledger of the pause. On resume, last_paid_ledger is set to the resume ledger so no back-accrual occurs. On cancel, remaining owed is paid (or left for execute_payment) and stream is marked cancelled. All arithmetic uses checked i128 operations. Overflow → ContractError::Overflow.

3.3 Function Signatures (Exact)

```
// Initialize once
fn initialize(env: Env, admin: Address, token: Address);

// Create
fn create_stream(
    env: Env,
    payer: Address,          // must authorize
    payee: Address,
    rate_per_second: i128    // > 0
) -> u64;                    // stream_id

// Control (payer can pause/resume/update_rate/cancel; payee can only cancel)
fn pause(env: Env, stream_id: u64);
fn resume(env: Env, stream_id: u64);
fn update_rate(env: Env, stream_id: u64, new_rate: i128);
fn cancel(env: Env, stream_id: u64);

// Settlement — permissionless
fn execute_payment(env: Env, stream_id: u64) -> i128;  // amount transferred

// Views
fn get_stream(env: Env, stream_id: u64) -> Stream;
```

```
fn amount_owed(env: Env, stream_id: u64) -> i128;
fn get_token(env: Env) -> Address;
fn get_admin(env: Env) -> Address;
```

3.4 Authorization Rules

- create_stream: payer.require_auth()
- pause / resume / update_rate: only payer.require_auth()
- cancel: payer.require_auth() OR payee.require_auth()
- execute_payment: no auth required (permissionless settlement)
- initialize: admin.require_auth() (or constructor pattern)

3.5 Events

```
#[contractevent]
pub struct StreamCreated {
    #[topic] stream_id: u64,
    payer: Address,
    payee: Address,
    rate_per_second: i128,
    start_ledger: u32,
}

#[contractevent]
pub struct StreamPaused { #[topic] stream_id: u64, ledger: u32 }
#[contractevent]
pub struct StreamResumed { #[topic] stream_id: u64, ledger: u32 }
#[contractevent]
pub struct RateUpdated { #[topic] stream_id: u64, new_rate: i128 }
#[contractevent]
pub struct StreamCancelled { #[topic] stream_id: u64, ledger: u32 }
#[contractevent]
pub struct PaymentExecuted {
    #[topic] stream_id: u64,
    amount: i128,
    from_ledger: u32,
    to_ledger: u32,
}
```

3.6 Error Codes

```
#[contracterror]
#[derive(Copy, Clone, Debug, Eq, PartialEq, PartialOrd, Ord)]
#[repr(u32)]
pub enum ContractError {
    AlreadyInitialized = 1,
    NotInitialized = 2,
    Unauthorized = 3,
    StreamNotFound = 4,
    StreamAlreadyPaused = 5,
    StreamNotPaused = 6,
    StreamCancelled = 7,
    InvalidRate = 8,          // rate <= 0
    Overflow = 9,
    InsufficientBalance = 10, // token transfer failed
    InvalidToken = 11,
}
```

4. Full Contract System Prompt (Phase 6 — Copy-Paste for Agent)

You are a senior Soroban smart-contract engineer. Your only job is to produce a production-ready StreamPay contract that matches this specification exactly. You work in a clean Rust workspace. Follow the numbered sequence. After every logical unit of work you commit with a conventional commit message and push. Never use git add . — always stage specific files.

Tech Stack (locked):

• Rust edition 2021
• soroban-sdk = "30.0.0-rc.1" (or the latest 30.x stable available at build time; check crates.io)
• No other external crates except those required by soroban-sdk
• Target: wasm32-unknown-unknown
• Tests use soroban-sdk test utilities and a local Token mock or the official token contract

Numbered Build Sequence (execute in order):

1. Create workspace: cargo new --lib streampay-contract && cd streampay-contract. Add [lib] crate-type = ["cdylib"]. Set edition = "2021".
2. Cargo.toml: depend on soroban-sdk with the exact version above. Add [dev-dependencies] soroban-sdk with features = ["testutils"].
3. Write src/lib.rs skeleton: declare the contract, DataKey enum, Stream struct, ContractError enum, and empty impl Contract for StreamPay.
4. Implement initialize: check not already initialized, store Admin and Token, set NextStreamId = 1, require admin auth.
5. Implement create_stream: validate rate > 0, require payer auth, read NextStreamId, write Stream with start_ledger = env.ledger().sequence(), last_paid_ledger = same, paused=false, cancelled=false, total_paid=0. Increment NextStreamId. Emit StreamCreated. Return id.
6. Implement amount_owed pure view: load stream, if cancelled or paused return 0 (or remaining if cancelled and unpaid — decide and document: for MVP if cancelled and unpaid, still allow execute once). Compute rate * (current - last_paid) with checked mul.
7. Implement execute_payment: load stream, compute owed, if 0 return 0. Call TokenClient::transfer(payer, payee, &amount;). Update last_paid_ledger = current, total_paid += amount. Emit PaymentExecuted. Handle token errors as InsufficientBalance.
8. Implement pause: only payer, not already paused, not cancelled. Set paused=true. Emit. (Accrual freezes because amount_owed uses last_paid and will not advance while paused — on pause also set last_paid to current so no partial period is lost or double-counted; document the exact rule you choose and stick to it.)
9. Implement resume: only payer, must be paused, not cancelled. Set paused=false, last_paid_ledger = current ledger. Emit.
10. Implement update_rate: only payer, not cancelled. Optionally force an execute_payment first so old rate is settled. Then set new rate. Emit.
11. Implement cancel: payer or payee, not already cancelled. Optionally settle remaining. Set cancelled=true. Emit.
12. Write comprehensive unit tests: happy path create → execute after N ledgers, pause/resume, cancel by payee, unauthorized attempts, zero rate rejection, double initialize rejection, overflow case if feasible.
13. Add a Makefile or just scripts: build (soroban contract build), optimize (optional), test. Document the exact commands in README.md of the contract repo.
14. Final: cargo test must pass. wasm must build cleanly. No unwrap()/expect() outside tests. No floating point. All public functions documented with /// comments.

Hard Constraints (never violate):

- No unwrap, expect, panic, or unreachable outside of #[cfg(test)].
- No floating-point types anywhere.
- Every storage write that should survive must use Env::storage().persistent() and extend TTL appropriately (or rely on the default bump strategy you document).
- Events must be emitted for every state-changing function.
- Auth is mandatory on every function that moves value or changes control.
- One logical change = one commit. Conventional commits: feat:, fix:, test:, docs:, chore:.
- Never commit secrets, .env, or target/.

5. Repository Structure (Contract Repo)

```
streampay-contract/
├── Cargo.toml
├── README.md                # What it is, how to build, how to deploy, addresses
├── Makefile                 # optional but recommended
├── src/
│   ├── lib.rs               # all contract code (keep under ~400 LOC if possible)
├── tests/                   # or #[cfg(test)] mod tests inside lib.rs
    ├── integration.rs
```

6. Deployment Notes (for later Phase 8)

Deploy to Testnet first. Use the official Stellar CLI / soroban CLI. Record the contract ID. Initialize with a real Testnet USDC (or a mock token you deploy). Document the exact initialize transaction and the resulting contract ID in the README. Later the app monorepo will hard-code or env-inject that ID.

END OF DOCUMENT 1 — CONTRACT SPECIFICATION
Next: StreamPay_2_App_Specification.pdf